Assignment: Build an AI-Powered Document Question-Answering System Using RAG

Module: RAG + AI-Powered Data Applications
Type: Practical, Hands-on Assignment
Objective: Build a Retrieval-Augmented Generation (RAG) application that answers questions from documents and provides source citations.

1. Problem Statement

Organizations store important information in PDFs, reports, technical documentation, employee handbooks, and policy documents. Finding specific information manually can be time-consuming.

Your task is to develop an **AI-powered Document Question-Answering System** that allows users to upload or process documents, ask questions in natural language, and receive answers based on the information available in those documents.

The system must retrieve relevant information before generating an answer and display the source documents used to formulate the response.

2. Technology Stack

Use the following technologies:

- **Programming language:** Python
- **Document processing:** PyPDF
- **Text splitting:** LangChain Text Splitters
- **Embeddings:** Sentence Transformers or another suitable embedding model
- **Vector database:** ChromaDB or FAISS
- **LLM:** Ollama with a suitable local model, or an available LLM API
- **Optional interface:** Streamlit or FastAPI

You may use alternative tools if you explain why they are appropriate.

3. Functional Requirements

Task 1: Document Ingestion

Create a document ingestion pipeline that can read PDF documents.

Your application should:

- Load multiple PDF files.
- Extract text from the documents.
- Handle empty pages or documents with missing text.
- Preserve metadata such as document name and page number.
- Report the number of documents or pages processed.

Task 2: Text Chunking

Split the extracted text into smaller chunks suitable for semantic search.

Your implementation should:

- Use a configurable chunk size.
- Implement chunk overlap.
- Preserve source metadata for every chunk.
- Display sample chunks for inspection.

Experiment with at least two different chunk sizes and observe how the results change.

Task 3: Generate Embeddings

Convert document chunks into vector embeddings using an embedding model.

Your implementation should:

- Generate embeddings for document chunks.
- Generate an embedding for a user query.
- Explain why embeddings are useful for semantic search.
- Store the generated document embeddings in a vector database.

Task 4: Vector Database and Retrieval

Store the chunks and their metadata in ChromaDB or FAISS.

When a user submits a question, the application should:

1. Process the question.
2. Search for semantically relevant chunks.
3. Retrieve the top-K results.
4. Display the retrieved text and its metadata.

5. Allow the number of retrieved chunks to be configured.

Task 5: LLM-Based Answer Generation

Pass the retrieved context and user question to an LLM.

The prompt should instruct the model to:

- Answer using the retrieved context.
- Avoid inventing information that is not supported by the context.
- Clearly indicate when the documents do not contain sufficient information.
- Provide a concise, understandable response.

Task 6: Source Citations

Every generated answer should include source information.

For example:

Question: What is the purpose of a vector database?

Answer: A vector database stores vector embeddings and supports similarity search over them.

Sources:

- `technical_documentation.pdf`, Page 4
- `database_notes.pdf`, Page 12

The example above is illustrative. Your application must cite the actual documents and pages from which the answer was retrieved.

Task 7: Metadata Filtering

Add metadata to your document chunks, such as:

- Document name
- Department or subject
- Document category
- Year, where applicable

Implement at least one metadata filter so that users can restrict retrieval to a selected document category, subject, or other available metadata field.

Explain how metadata filtering changes the retrieval results.

Task 8: Retrieval Quality Evaluation

Create a small evaluation dataset containing at least 10 questions based on your documents.

For every question, record:

- The question asked
- The expected relevant document or passage
- The retrieved documents or chunks
- Whether the retrieved context is relevant
- Whether the generated answer is supported by the retrieved context

Include examples of questions for which retrieval succeeds and questions for which retrieval fails.

Calculate retrieval precision and recall wherever you have sufficient relevance labels.

Use the following formulas:

Precision

Precision = Relevant retrieved chunks / Total retrieved chunks

Recall

Recall = Relevant retrieved chunks / Total relevant chunks in the evaluation dataset

Clearly define what counts as relevant and ensure that the total relevant chunks are labelled for each evaluation question. Do not treat the percentage of questions with a correct source as equivalent to precision or recall.

4. Experiments and Observations

Perform the following experiments:

Experiment A: Chunk Size

- Run retrieval using two different chunk sizes.
- Compare the retrieved passages.
- Identify which configuration provides more useful context.

Experiment B: Top-K Retrieval

- Compare at least two values of K, such as K = 3 and K = 5.
- Observe changes in relevance and context completeness.
- Identify cases where retrieving more chunks introduces irrelevant information.

Experiment C: Retrieval vs Answer Quality

- Identify one example where the retriever returns irrelevant information.
- Identify one example where the retrieved context is relevant but the generated answer is incorrect or incomplete, if encountered.
- Explain the likely cause of each failure.

Experiment D: Metadata Filtering

- Run a query with and without metadata filtering.
- Compare the retrieved results and explain the difference.

5. Suggested Application Interface

If you build a Streamlit or web interface, consider including:

- A document selection or upload area
- A question input box
- An option to configure Top-K
- A metadata filter
- A generated answer section
- A retrieved-context section
- Source citations with page numbers

A command-line application is also acceptable if it implements the required functionality.

6. Expected Deliverables

Submit the following:

1. **Source code:** All Python files required to run the application.
2. **Requirements file:** A `requirements.txt` file containing the dependencies.
3. **Sample documents:** A small set of documents used for testing, provided you have permission to share them.
4. **README:** Setup instructions, execution steps, architecture, and technology choices.
5. **Evaluation dataset:** At least 10 questions with expected relevant passages or sources and observed retrieval results.
6. **Experiment report:** Observations about chunk size, Top-K, metadata filtering, and retrieval failures.
7. **Demonstration:** Screenshots or a short recording showing document ingestion, question answering, retrieved context, and source citations.

Do not include API keys, passwords, or other secrets in your submission.

7. Suggested Implementation Order

Follow this sequence while developing your application:

1. Load and inspect the documents.
2. Extract text and preserve metadata.
3. Split documents into chunks.
4. Generate embeddings.
5. Store chunks in a vector database.
6. Implement semantic retrieval.
7. Connect the LLM.
8. Generate answers using retrieved context.
9. Add source citations.
10. Implement metadata filtering.
11. Evaluate retrieval precision and recall.
12. Experiment with chunk size and Top-K.
13. Document your findings.

8. Questions to Address in Your Report

1. What problem does RAG solve?
2. How is RAG different from ordinary prompting?
3. Why do we convert text into embeddings?
4. What is the role of a vector database?
5. Why is chunk overlap useful?
6. How does metadata filtering improve retrieval?
7. What is the difference between retrieval precision and recall?
8. Can a RAG application produce an incorrect answer even when retrieval is successful? Explain.
9. What happens when the relevant information is absent from the documents?
10. Which configuration performed best in your experiments, and what evidence supports your conclusion?

9. Completion Criteria

Your assignment is complete when you can demonstrate that the application:

- Processes multiple documents.
- Generates and stores embeddings.
- Retrieves relevant context for user questions.
- Produces answers grounded in retrieved information.
- Displays source citations.
- Supports metadata filtering.
- Evaluates retrieval quality using a labelled question set.
- Explains its limitations and experimental findings.

Final objective: Build a working RAG application and demonstrate that you understand the complete pipeline—from document ingestion to retrieval evaluation—not merely how to call an LLM API.